

L1 - Encapsulation and Polymorphism:

1.1 Encapsulation:

Definition:

Encapsulation is the bundling of data (variables) and the methods operating on that data into a single unit (class), while restricting direct external access to the internal state.

Mechanism in Java:

Access Modifiers:

Variables are marked as private to prevent external classes from reading or modifying them directly.

Getter/Setter Methods:

Public methods act as controlled gateways, allowing validation and read/write control over private fields.

Real-Life Analogy:

An ATM machine; You interact with a public interface (buttons/screen) to deposit or view cash, but you cannot directly touch or modify the internal vault (private data).

1.2 Polymorphism:

Definition:

Polymorphism allows objects or methods to behave differently based on the execution context or input parameters.

Distinguishing compile-time (static) vs run-time (dynamic) polymorphism:

Feature	Compile-time	Run-time
Mechanism	1. Method Overloading (Same method name, different parameters).	1. Method Overriding (Subclass redefines a superclass method).
Resolution	2. Resolved during compilation.	2. Resolved at run-time via dynamic binding.
Example	<pre> class Calculator { int add(int a, int b) { return a+b; } double add(double a, double b) { return a+b; } } </pre>	<pre> class Animal { void sound() { System.out.println ("Animal sound"); } } class Dog extends Animal { @Override void sound() { System.out.println ("Bark"); } } </pre>

1.3 Source code:

```

class BankAccount {
    private double balance;

    public BankAccount (double initialBalance) {
        if (initialBalance >= 0) {
            this.balance = initialBalance;
        }
        else {
            this.balance = 0.0;
        }
    }

    public double getBalance () {
        return balance;
    }

    public void deposit (double amount, String
                        remarks) {
        if (amount > 0) {
            balance += amount;

            System.out.println ("Successfully deposited: $"
                                + amount + " | Note: " + remarks);
        }
        else {
            System.out.println ("Invalid deposit amount.");
        }
    }
}

```



```
public class Main {  
    public static void main (String[] args) {  
        BankAccount account = new BankAccount (500.0);  
        System.out.println ("Initial Balance: $" +  
            account.getBalance ());  
        account.deposit (150.0);  
        System.out.println ("Current Balance: $" +  
            account.getBalance ());  
        account.deposit (200.0, "Monthly Savings");  
        System.out.println ("Final Balance: $" +  
            account.getBalance ());  
    }  
}
```

L-2: Method Overloading vs Overriding (Early vs Late Binding)

[2.1] Comparing Overloading and Overriding:

Feature	Overloading	Overriding
Definition	Multiple methods in the same class with the same name but different parameter lists.	A base class method in a subclass with the exact same signature.
Class Involved	Occurs within a single class.	Requires at least two classes in an inheritance relationship (Parent-child).
Parameters	Must be different (number, type, or sequence of parameters).	Must be identical (same number, type and order of parameters).
Return Type	Can be same or different.	Must be the same (or a covariant subtype).
	Resolved at Compile-Time (Static/Early Binding).	Resolved at Run-Time (Dynamic Late Binding).

2.2 Early Binding:

The compiler links a method call to its method body during compilation before the program runs. (Also called Static Binding)

Late Binding (Dynamic):

The JVM links a method call to its corresponding method implementation at runtime based on the actual object type in memory.

Overloading is resolved at compile time. As it depends strictly on the method signature (method name and parameter list).

Since parameter types and counts are fully known to the compiler when parsing the code, the compiler determines exactly which overloaded method to execute directly during compilation.

Overriding is resolved at Run Time.

As it depends on the actual runtime object type assigned to a parent reference variable. The compiler only

checks that the parent reference has the method declared, but the JVM

inspects the heap memory at runtime to execute the subclass's overridden implementation dynamically via virtual method invocation.

2.3 Source Code:

```

class Shape {
    public double area() {
        return 0.0; }

    public void describe (String name) {
        System.out.println ("Shape Name: " + name);
    }

    public void describe (String name, int sides) {
        System.out.println ("Shape Name: " + name +
            " | Number of sides: " + sides);
    }
}

class Circle extends Shape {
    private double radius;

    public Circle (double radius) {
        this.radius = radius;
    }

    @Override
    public double area () {
        return Math.PI * radius * radius;
    }
}

class Rectangle extends Shape {
    private double length;
    private double width;
}

```

```
public Rectangle (double length, double width) {
    this.length = length;
    this.width = width;
}
```

```
@Override
public double area () {
    return length * width;
}
```

```
public class Main {
    public static void main (String[], args) {
        Shape baseShape = new Shape ();
        System.out.println ("Overloading (Early Binding)");
        baseShape.describe ("Circle");
        baseShape.describe ("Rectangle", 4);
    }
}
```

```
System.out.println ("Overriding (Late Binding)");
Shape shape1 = new Circle (5.0);
System.out.println ("Circle Area: %.2f", shape1.area());
Shape shape2 = new Rectangle (4.0, 6.0);
System.out.println ("Rectangle Area: %.2f", shape2.
    area ());
}
```


L3 - Abstract Class vs Interface:

3.1 Abstract Class:

A restricted class declared with the abstract keyword that cannot be instantiated directly. It serves as a partial blueprint for subclasses, allowing a mix of abstract methods (without bodies) and concrete methods (with bodies).

Interface:

A blueprint of a class that defines a contract/capability. In Java, it specifies what a class must do without defining how, containing abstract methods (by default) and static/final constants.

Structural Differences:

Aspect	Abstract Class	Interface
Fields	Can have instance variables of any access level (private, protected, public, static, final, non-final).	Fields are implicitly public static final constants only.
Constructors	Can have constructors	Cannot have constructors
Method Bodies	Can contain both concrete methods (with implementation) and abstract methods.	All methods are abstract act by default (except default and static).
Multiple Inheritance	A class can extend only one abstract class (Single Inheritance).	A class can implement multiple interfaces.

Use abstract class when creating closely related classes that share a common non-identity and need to share common non-static fields or reusable concrete method implementations.

Real-World Scenario: A vehicle parent class for Car, Truck, and Motorcycle. All vehicles share common state (brand, speed) and behavior (startEngine()), but each specific type implements its own fuelType().

Use interface when defining a contract or capability that can be applied across completely unrelated classes without imposing a strict parent-child hierarchy.

Real-World Scenario: An Insurable interface defining calculatePremium(). A Car, a House, and a Commercial ship belong to totally different class hierarchies, but all three possess the "can-be-insured" capability and implement calculatePremium().

3.3 Source Code:

```

abstract class Vehicle {
    public void startEngine () {
        System.out.println("Engine started successfully.");
    }
    public abstract String fuelType();
}

interface Insurable {
    double calculatePremium ();
}

class Car extends Vehicle implements
    Insurable {
    private double markedValue;
    public Car (double markedValue) {
        this.markedValue = markedValue;
    }
    @Override
    public String fuelType () {
        return "Octane/Gasoline";
    }
    @Override
    public double calculatePremium () {
        return markedValue * 0.05;
    }
}

```



```

public class Main {
    public static void main (String[] args) {
        Car myCar = new Car (25000.0);

        System.out.println ("--- Car Operations ---");

        myCar.startEngine();

        System.out.println ("Fuel Type: " +
            myCar.fuelType());

        System.out.println ("Annual Insurance
            Premium: $%.2f\n", myCar.calculateP-
                remium());
    }
}

```

- Vehicle (Abstract class): Chosen for an "is-a" relationship to share concrete code (startEngine()) and enforce subclass implementations (fuelType()).
- Insurable (Interface): Chosen for a "can-do" capability to allow unrelated entities (Car, House) to calculate premiums without restricting inheritance.

L4- Collection Framework:

4.1 ArrayList vs Vector vs Linked List:

Feature	ArrayList	Vector	Linked List
Structure	Resizable array	Resizable array	Doubly Linked List
Synchronized (thread-safe)	No	Yes	No
Random access (get by index)	Fast	Fast	slow (must walk the chain)
Insert/delete (middle)	Slow (shifts elements)	Slow	Fast (just re-links pointers)

4.2 Set and its implementation:

A set is a collection that does not allow duplicate elements.

HashSet: No guaranteed order, fastest.

LinkedHashSet: Keeps insertion order.

TreeSet: Keeps elements sorted automatically. It maintains order using a Red-Black Tree internally - every time you add an element, it's placed in the correct sorted position (using natural ordering via Comparable, or a custom Comparator).

4.3 Source code:

```
import java.util. ArrayList;
import java.util. TreeSet;
```

```
public class Main {
```

```
public static void main (String[] args) {
```

```
ArrayList<String> list = new ArrayList<> ();
```

```
list.add ("David");
```

```
list.add ("Alice");
```

```
list.add ("Ethan");
```

```
list.add ("Bob");
```

```
list.add ("Carol");
```

```
System.out.println ("ArrayList (insertion
```

```
order: ");
```

```
for (String name: list) { System.out.println  
(name); } } TreeSet<String> set = new TreeSet<>(list);  
System.out.println ("\nTreeSet (sorted
```

```
order): ");
```

```
for (String name: set) {
```

```
System.out.println (name);
```

```
}
```

```
}
```

```
}
```


L5 - Multithreading & Custom Exception Handling:

5.1 Ways to implement multithreading:

1. Extending Thread and overriding run().
2. Implementing Runnable and passing it to a Thread object.
3. Using ExecutorService/Callable - a thread pool manages threads for you, and Callable can return a result. Preferred! Runnable (or ExecutorService) is generally preferred, because Java only allows single inheritance - if your class extends Thread, it can't extend anything else. Runnable keeps your class free to extend other classes, and separates "the task" from "the thread that runs it".

5.2 Two threads:

```

class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread (extends): " + i);
        }
    }
}

```

```
class MyRunnable implements Runnable {  
    public void run () {  
        for (int i=1; i<=5; i++) {  
            System.out.println ("Thread (runnable): " + i);  
        }  
    }  
}
```

```
public class Main {  
    public static void main (String[] args) {  
        MyThread t1 = new MyThread ();  
        Thread t2 = new Thread (new MyRunnable());  
        t1.start ();  
        t2.start ();  
    }  
}
```

5.3 Source code:

```

class InvalidRadiusException extends Exception {
public InvalidRadiusException (String msg) {
    super (msg);
}
}

class Circle {
    private double radius;

    public Circle (double radius) throws InvalidRadiusException {
        if (radius < 0) {
            throw new InvalidRadiusException ("Radius
            cannot be negative; " + radius);
        }
        this.radius = radius;
    }

    public double area () {
        return Math.PI * radius * radius;
    }
}

import java.util.Scanner;

```



```
public class Main {  
    public static void main (String[] args) {  
        Scanner sc = new Scanner (System.in);  
        System.out.print ("Enter radius: ");  
        double r = sc.nextDouble();  
        try {  
            Circle c = new Circle(r);  
            System.out.println ("Area: " + c.area());  
        } catch (InvalidRadiusException e) {  
            System.out.println ("Error: " + e.getMessage());  
        }  
    }  
}
```

LG - JDBC with MySQL/Oracle (MVC Pattern)°

[6.1] steps to connect Java to a database

via JDBC:

1. Load/Register the JDBC driver
(usually automatic since JDBC 4.0)

2. Open a connection:

```
Connection con = DriverManager.getConnection  
(url, user, password);
```

3. Create a statement; PreparedStatement
ps = con.prepareStatement(sql);

4. Run the query/update; ps.executeQuery()
or ps.executeUpdate().

5. Process results using ResultSet (for
SELECT queries).

6. Close the connection. Key classes:

DriverManager (creates connections), Connection
(represents the DB session), PreparedStatement.
Statement (runs safe, parameterized SQL),
ResultSet (holds query results).

6.2 MVC pattern in a JDBC app: MVC splits an app into three parts:

- Model - the plain data class (e.g. Student) representing one record.
- View - whatever collects input and shows output to the user (e.g. Main, or a JSP page).
- Controller - the class that handles logic and talks to the database (e.g. StudentDAO), sitting between Model and View.

6.3 Source code:

```
import java.sql.*;
import java.util.*;
```

```
class Student {
    private int id;
    private String name;
    private double cgpa;
```

```
public Student (int id, String name, double cgpa) {
    this.id = id; this.name = name;
    this.cgpa = cgpa; }
```



```

public int getId() { return id; }

public String getName() { return name; }

public double getGpa() { return gpa; }
}

```

```

class StudentDAO {

    public void insert(Student s) throws
    SQLException {

```

```

        String url = "jdbc:mysql://localhost:
        3306/student_db";

```

```

        try (Connection con = DriverManager.getConnection(url, "root", "password");

```

```

        PreparedStatement ps = con.prepareStatement
        ("INSERT INTO Students
        (id, name, gpa) VALUES (?, ?, ?))");

```

```

        ps.setInt(1, s.getId());
        ps.setString(2, s.getName());
        ps.setDouble(3, s.getGpa());
        ps.executeUpdate();
    }
}
}

```

```
public class Main {  
    public static void main (String[] args)  
        throws SQLException {  
        Scanner sc = new Scanner(System.in);  
        System.out.print ("ID: ");  
        int id = sc.nextInt();  
        System.out.print ("Name: ");  
        String name = sc.next();  
        System.out.print ("CGPA: ");  
        double cgpa = sc.nextDouble();  
        Student s = new Student (id, name, cgpa);  
        new StudentDAO (). insert (s);  
        System.out.println ("Student saved!");  
    }  
}
```

L7 - JavaFX - House Loan Calculator.

[7.1] JavaFX application structure:

- Application - the base class every JavaFX app extends; JavaFX calls its start() method to launch the GUI.
- Stage - the actual window on screen.
- Scene - the content that fills the window (like a canvas holding all your UI elements).
- Gridpane / VBox - layout containers that arrange buttons, labels, and text fields (grid-style vs vertical-stack style). Role of start(): It's the entry point JavaFX calls automatically after launch - this is where you build the scene, set up the layout, add event handlers, and show the Stage.

7.2 GridPane layout:

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;
```

```
public class LoanCalculator extends Application {
```

```
    TextField loanField = new TextField();
```

```
    TextField rateField = new TextField();
```

```
    TextField yearsField = new TextField();
```

```
    Label resultLabel = new Label();
```

```
    @Override
```

```
    public void start(Stage stage) {
```

```
        GridPane grid = new GridPane();
```

```
        grid.setPadding(new Insets(10));
```

```
        grid.setHgap(10);
```

```
        grid.setVgap(10);
```

```
        grid.add(new Label("Loan Amount: "), 0, 0);
```

```
        grid.add(loanField, 1, 0);
```

```
        grid.add(new Label("Annual Rate(%): "), 0, 1);
```

```
        grid.add(rateField, 1, 1);
```

```

grid.add(new Label("Number of Years:"), 0, 2);
grid.add(yearsField, 1, 2);

Button calcBtn = new Button("Calculate");
grid.add(calcBtn, 1, 3);

grid.add(resultLabel, 0, 4, 2, 1);
calcBtn.setOnAction(e -> calculate());

stage.setScene(new Scene(grid, 350, 250));
stage.setTitle("House Loan Calculator");

stage.show();
}

private void calculate() {
    double P = Double.parseDouble(loanField.getText());
    int n = Integer.parseInt(yearsField.getText())
        * 12;

    double r = annualRate / 12 / 100;
    double M = P * r * Math.pow(1+r, n) /
        (Math.pow(1+r, n) - 1);

    double T = M * n;
    double D = T - P;

```

```

resultLabel.setText (String.format (
    "Monthly: %.2f | Total: %.2f |
    Difference: %.2f ", M, T, D));
}

public static void main (String[] args) {
    launch (args);
}
}

```

7.3 Event handler (already included above):

The `calcBtn.setOnClickListener (e → calculate ())` line runs the `calculate ()` method when the button is clicked. It uses the standard ~~and~~ loan amortization formula:

$$r = \text{AnnualRate} / 12 / 100$$

$$M = P \times r \times (1+r)^n / [(1+r)^n - 1]$$

$$T = M \times n$$

$$D = T - P$$

Then it updates `resultLabel` with the Monthly Installment, Total Payment, and Difference.

L8- Socket Programming & Java RMI (Chat System)

8.1 Socket vs Java RMI:

Sockets: low-level, you send and receive raw text/bytes yourself over a socket/ServerSocket connection. You control the exact message format.

RMI (Remote Method Invocation): high-level, you call a method on a remote object almost like its local - Java handles the network communication, or a lightweight custom messaging (like a chat app). Use RMI when you're working purely in Java and want to call remote methods directly without manually building a message protocol.

8.2 Server Code:

```
import java.io.*;
import java.net.*;

public class ChatServer {
    public static void main (String[] args)
        throws IOException {
```

```
ServerSocket serverSocket = new
    ServerSocket (5000);

System.out.println ("Server waiting for
    connection...");

Socket socket = serverSocket.accept();

System.out.println ("Client connected!");

BufferedReader in = new BufferedReader
    (new InputStreamReader (socket.getInputStream()
        read ()));

PrintWriter out = new PrintWriter (socket.getOutputStream()
    writeStream (), true);

String message = in.readLine();

System.out.println ("Client says: " + message);

System.out.println ("Server received: " +
    message);
socket.close();
serverSocket.close();
}
```

8.3 Client Code:

```
import java.io.*;
import java.net.*;

public class ChatClient {
    public static void main(String[] args)
        throws IOException {
        Socket socket = new Socket("localhost",
            5000);
```

```
    PrintWriter out = new PrintWriter(socket,
        getOutputStream(), true);
    BufferedReader in = new BufferedReader
        (new InputStreamReader(socket.getInputStream()
        ));
```

```
    out.println("Hello Server!");
    String reply = in.readLine();
    System.out.println("Server reply: " + reply);
    socket.close();
}
```

}

L9- Servlet + JSP + JDBC CRUD (Student Records):

[9.1] Steps to set up the project:

1. Create the MySQL database student -db and a Students table with columns id, name, gpa.
2. Set up a Dynamic Web Project (or Maven web app) in your IDE, add the MySQL JDBC connector .jar to the classpath.
3. Map the Servlet to a URL pattern (e.g. /addStudent) using @WebServlet or web.xml.
4. Build the JDBC connection string, e.g. jdbc:mysql://localhost:3306/student-db.
5. Write the Servlet (handles form submissions), a DAO class (does the DB work), and a JSP page (shows results).

9.2 Servlet doPost()

```
import java.io.*;
import java.sql.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
public class AddStudentServlet extends
```

```
HttpServlet {
    protected void doPost(HttpServletRequest
    request, HttpServletResponse response)
    throws ServletException, IOException {
        int id = Integer.parseInt(request.getParameter("id"));
    }
```

```
String name = request.getParameter("name");
double cgpa = Double.parseDouble(request.
    getParameter("cgpa"));
```

```
String url = "jdbc:mysql://localhost:
    3306/student_db";
```

```
try (Connection con = DriverManager.getConnection(
    url, "root", "password");
```

```
PreparedStatement ps = con.prepareStatement(
    "INSERT INTO Students (id,
    name, cgpa) VALUES (?, ?, ?)"); {
```

```
ps. setInt (1, id);  
ps. setString (2, name);  
ps. setDouble (3, gpa);  
ps. executeUpdate ();  
  
response.getWriter ().println ("Student added  
successfully!");  
} catch (SQLException e) {  
  
response.getWriter ().println ("Error:"  
+ e.getMessage ());  
}  
}  
}
```


9.3 JSP page:

```

<%@ page import = "java.sql.*"%>
<html>
<body>
<h2> Student List </h2>
<table border = "1">
<tr><th> ID </th><th> Name </th>
<th> CGPA </th> </tr>
<%

```

```

    connection con = DriverManager.getConnection (
    "jdbc:mysql://localhost:3306/student_
    db", "root", "password");

```

```

Statement st = con.createStatement();
ResultSet rs = st.executeQuery ("SELECT
* FROM students");
while (rs.next()) {
    %>
<tr>
<td> <%= rs.getInt ("id") %> </td>

```

```
<td> <% = rs.getString("name") %> </td>  
<td> <% = rs.getDouble("gpa") %> </td>
```

```
</tr>
```

```
<%
```

```
{
```

```
    con.close();
```

```
%>
```

```
</table>
```

```
</body>
```

```
</html>
```